

복합문제 (1) - i-node 읽기 개수와 순서

풀이

Unix 파일 시스템에서 파일을 열려면, 경로명을 따라 각 디렉터리/파일의 i-node를 순서대로 탐색해야 합니다.

순서 1. / (루트 디렉터리) i-node

순서 2. /usr 디렉터리 i-node

순서 3. /usr/dev 디렉터리 i-node

순서 4. /usr/dev/source 디렉터리 i-node

순서 5. /usr/dev/source/app.c 파일의 i-node

총 5개의 i-node를 읽어야 한다.

핵심 원리: 각 디렉터리의 i-node → 디렉터리 블록 읽기 → 다음 경로 구성요소의 i-node 번호 획득 → 반복

복합문제 (2) - 디스크 접근 순서 (i-node + 블록 모두 포함)

풀이

파일을 열고(open) 실제 데이터까지 읽으려면 i-node 뿐 아니라 각 디렉터리의 실제 블록도 읽어야 합니다.

순서 1. 루트(/) i-node

순서 2. 루트 디렉터리 블록들

순서 3. /usr의 i-node

순서 4. /usr 디렉터리 블록

순서 5. /usr/dev의 i-node

순서 6. /usr/dev 디렉터리 블록

순서 7. /usr/dev/source의 i-node

순서 8. /usr/dev/source 디렉터리 블록

순서 9. /usr/dev/source/app.c의 i-node

순서 10. /usr/dev/source/app.c의 파일 블록 (실제 데이터)

총 10회 디스크 접근 발생

핵심 원리:

경로 탐색 시 i-node만으로는 부족, 디렉터리 블록도 읽어야 다음 항목의 i-node 번호를 알 수 있음

마지막에 파일 데이터 블록까지 읽어야 실제 read() 완료

복합문제 (3) - 파일 open() 시 i-node를 메모리에 올리는 이유

풀이

i-node에는 파일 크기, 파일 블록의 위치 정보, 접근 권한 등 핵심 메타 정보가 저장되어 있습니다.

파일 읽기(read())나 쓰기(write())가 진행되는 동안 이 메타 정보는 반복적으로 참조됩니다.

만약 i-node를 디스크에 그대로 둔다면:

파일 블록을 읽을 때마다 i-node도 디스크에서 반복 읽기 → 매우 느림

디스크 I/O 횟수 급증 → 성능 저하

따라서 파일을 열 때 i-node를 메모리(i-node 테이블)에 미리 올려두면:

이후 read()/write() 시 메모리에서 빠르게 참조 가능

디스크 접근 횟수를 최소화 → 성능 향상

결론: 파일 입출력 시 반복 참조되는 i-node를 매번 디스크에서 읽는 것은 비효율적이므로, open() 시점에 메모리 i-node 테이블에 캐싱하여 빠른 접근을 가능하게 한다.